

Window Functions in Spark

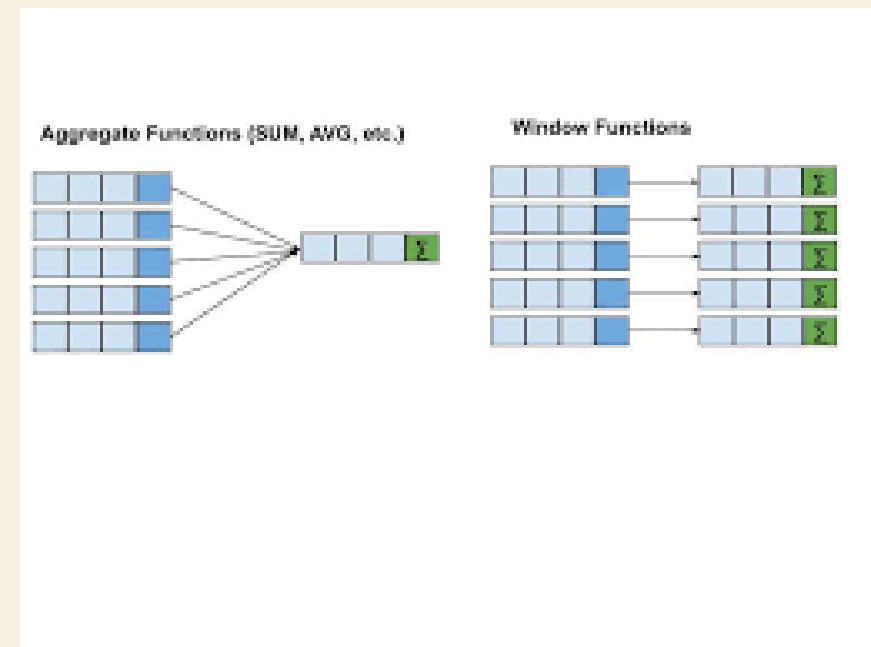


Window Functions in Spark

- Window functions are used to perform operations on a set of rows related to the current row.
- They are useful for tasks like ranking, cumulative sums, moving averages, etc.
- This is similar to SQL window functions which are used with the **OVER** clause.
- operate on a group of rows, referred to as a window, and calculate a return value for each row based on the group of rows.
- To perform window function operation on a group of rows first, we need to partition i.e. define the group of data rows using **window.partition()** function, and for row number and rank function we need to additionally order by on partition data using **ORDER BY** clause.

There are mainly three types of Window function:

- Analytical Function
- Ranking Function
- Aggregate Function



Types of window function

Window Function in SQL



```
%sql
SELECT id, name, score,
       RANK() OVER (ORDER BY score DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY score DESC) AS dense_rank,
       ROW_NUMBER() OVER (ORDER BY score DESC) AS row_number
FROM table_name;
```

There are mainly three types of Window function:

1. Analytical Functions:

- Description:** These functions perform calculations across a set of table rows that are somehow related to the current row.
- Examples:** `LEAD()`, `LAG()`, `FIRST_VALUE()`, `LAST_VALUE()`
- Use Case:** Useful for accessing data from a subsequent or preceding row in the same result set without the need for a self-join.

2. Ranking Functions:

- Description:** These functions assign a rank to each row within a partition of a result set.
- Examples:** `RANK()`, `DENSE_RANK()`, `ROW_NUMBER()`
- Use Case:** Useful for ranking rows within a partition, such as finding the top N records.

3. Aggregate Functions:

- Description:** These functions perform a calculation on a set of values and return a single value.
- Examples:** `SUM()`, `AVG()`, `MIN()`, `MAX()`
- Use Case:** Useful for performing calculations like sums, averages, and counts over a window of rows.

Window Functions

Aggregate

- `AVG()`
- `MAX()`
- `MIN()`
- `COUNT()`

Ranking

- `RANK()`
- `DENSE_RANK()`
- `ROW_NUMBER()`
- `PERCENT_RANK()`
- `NTILE()`

Value

- `LAG()`
- `LEAD()`
- `CUME_DIST()`

Window Functions in Spark

Window Specification (WindowSpec)

A **WindowSpec** defines the partitioning, ordering, and frame boundaries for the window function. It is created using the **Window** class.

Syntax for WindowSpec :

```
from pyspark.sql import Window
```

```
window_spec= Window.partitionBy ("column_name").orderBy ("column_name")
```

Syntax for Window function:

```
df = df.withColumn ("new_col _name", Window_function() .over( window_spec))
```



Window_function() –

- LEAD (), LAG (), FIRST_VALUE (), LAST_VALUE () .
- RANK (), DENSE_RANK (), ROW_NUMBER () .
- SUM (), AVG (), MIN (), MAX () .

ANALYTICAL FUNCTIONS IN PYSPARK

Analytical functions are used to perform calculations across a set of table rows that are somehow related to the current row. Here are some commonly used analytical functions:

1. LEAD:

1. **Description:** The `LEAD` function provides access to a subsequent row at a specified physical offset.
2. **Use Case:** Useful for comparing values in the current row with values in subsequent rows.

2. LAG:

1. **Description:** The `LAG` function provides access to a previous row at a specified physical offset.
2. **Use Case:** Useful for comparing values in the current row with values in preceding rows.

3. FIRST_VALUE:

1. **Description:** The `FIRST_VALUE` function returns the first value in an ordered set of values.
2. **Use Case:** Useful for retrieving the first value in a partition.

4. LAST_VALUE:

1. **Description:** The `LAST_VALUE` function returns the last value in an ordered set of values.
2. **Use Case:** Useful for retrieving the last value in a partition.

Ranking Function

1. Rank:

1. **Description:** The `rank` function assigns a rank to each row within a partition of the result set. The rank is based on the order specified in the `WindowSpec`. If two rows have the same value, they will receive the same rank, and the next rank will have a gap.
2. **Example:** If two rows are tied for rank 1, the next rank assigned will be 3 (i.e., there is a gap).

2. DenseRank:

1. **Description:** The `dense_rank` function is similar to `rank`, but it does not leave gaps in the ranking sequence. If two rows have the same value, they will receive the same rank, and the next rank will be consecutive.
2. **Example:** If two rows are tied for rank 1, the next rank assigned will be 2 (i.e., no gap).

3. RowNumber:

1. **Description:** The `row_number` function assigns a unique number to each row within a partition of the result set. The numbering is based on the order specified in the `WindowSpec`.
2. **Example:** Each row will have a unique number, starting from 1 for the first row in each partition.

EXAMPLE

Analytical Functions

```
from pyspark.sql import Window
from pyspark.sql.functions import lead, lag, first, last, col
# Sample data
data = [
    (1, "Alice", 50),
    (2, "Bob", 45),
    (3, "Charlie", 50),
    (4, "David", 40),
    (5, "Eve", 55)
]
columns = ["id", "name", "score"]
df = spark.createDataFrame(data, columns)
# Define WindowSpec
window_spec = Window.partitionBy().orderBy(col("score").desc())
# Apply analytical functions
df = df.withColumn("lead_score", lead("score", 1).over(window_spec)) \
    .withColumn("lag_score", lag("score", 1).over(window_spec)) \
    .withColumn("first_score", first("score").over(window_spec)) \
    .withColumn("last_score", last("score").over(window_spec))
df.show()
```



Output

```
df: pyspark.sql.dataframe.DataFrame = [id: long, name: string ... 5 more fields]
+---+-----+-----+-----+-----+-----+
| id | name | score | lead_score | lag_score | first_score | last_score |
+---+-----+-----+-----+-----+-----+-----+
| 5 | Eve | 55 | 50 | NULL | 55 | 55 |
| 1 | Alice | 50 | 50 | 55 | 55 | 50 |
| 3 | Charlie | 50 | 45 | 50 | 55 | 50 |
| 2 | Bob | 45 | 40 | 50 | 55 | 45 |
| 4 | David | 40 | NULL | 45 | 55 | 40 |
+---+-----+-----+-----+-----+-----+-----+
```

- **Lead** : Looks forward by 1 row(Eve next score=Alice(50))
- **Lag** : Looks backward by 1 row(Alice previous score=Eve(55))
- **First** : Eve's Score(55) for all rows
- **Last** : Last score David(40) for all rows

Note : last("score") doesn't return the actual last value by default because it only looks up to the current row. To get the correct result, use `.rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)` — try it out yourself!

EXAMPLE

Ranking Functions

```
%python
from pyspark.sql import Window
from pyspark.sql.functions import rank, dense_rank, row_number, col
# Sample data
data = [
    (1, "Alice", 50),
    (2, "Bob", 45),
    (3, "Charlie", 50),
    (4, "David", 40),
    (5, "Eve", 55)
]
columns = ["id", "name", "score"]
df = spark.createDataFrame(data, columns)
# Define WindowSpec
window_spec = Window.partitionBy().orderBy(col("score").desc())
# Apply ranking functions
df = df.withColumn("rank", rank().over(window_spec)) \
    .withColumn("dense_rank", dense_rank().over(window_spec)) \
    .withColumn("row_number", row_number().over(window_spec))
df.show()
```

Output

id	name	score	rank	dense_rank	row_number
5	Eve	55	1	1	1
1	Alice	50	2	2	2
3	Charlie	50	2	2	3
2	Bob	45	4	3	4
4	David	40	5	4	5



EXAMPLE

Aggregate Functions

Output

```
%python
from pyspark.sql import Window
from pyspark.sql.functions import sum, avg, min, max, col
# Sample data
data = [
    (1, "Alice", "Math", 85),
    (2, "Bob", "Math", 78),
    (3, "Charlie", "Math", 92),
    (4, "David", "Science", 88),
    (5, "Eve", "Science", 95),
    (6, "Frank", "Science", 80)
]
columns = ["id", "name", "subject", "score"]
df = spark.createDataFrame(data, columns)
# Define WindowSpec
window_spec = Window.partitionBy("subject").orderBy(col("score").desc
())
# Apply aggregate functions
df = df.withColumn("sum_score", sum("score").over(window_spec)) \
    .withColumn("avg_score", avg("score").over(window_spec)) \
    .withColumn("min_score", min("score").over(window_spec)) \
    .withColumn("max_score", max("score").over(window_spec))
df.show()
```



id	name	subject	score	sum_score	avg_score	min_score	max_score
3	Charlie	Math	92	92	92.0	92	92
1	Alice	Math	85	177	88.5	85	92
2	Bob	Math	78	255	85.0	78	92
5	Eve	Science	95	95	95.0	95	95
4	David	Science	88	183	91.5	88	95
6	Frank	Science	80	263	87.66666666666667	80	95

Note: It's showing cumulative values because `.orderBy()` with window applies default `rowsBetween(unboundedPreceding, currentRow)`—to fix it, `remove .orderBy()` or add `.rowsBetween(unboundedPreceding, unboundedFollowing)`.